

AI Lab Experiments with Questions and Easy Codes

Question: 1. Implement Breadth First Search (BFS) algorithm using Python.

```
from collections import deque

graph={'A':['B','C'],'B':['D','E'],'C':['F'],'D':[],'E':[],'F':[]}

def bfs(start):
    visited=set()
    queue=deque([start])
    visited.add(start)
    while queue:
        node=queue.popleft()
        print(node,end=" ")
        for n in graph[node]:
            if n not in visited:
                visited.add(n)
                queue.append(n)

bfs('A')
```

Question: 2. Implement Depth First Search (DFS) algorithm using Python.

```
graph={'A':['B','C'],'B':['D','E'],'C':['F'],'D':[],'E':[],'F':[]}
visited=set()
def dfs(node):
    if node not in visited:
        print(node,end=" ")
        visited.add(node)
        for n in graph[node]:
            dfs(n)
dfs('A')
```

Question: 3. Implement Uniform Cost Search (UCS) algorithm using Python.

```
import heapq
graph={'A': [('B',1), ('C',4)], 'B': [('D',2), ('E',5)], 'C': [('F',3)], 'D': [], 'E': [], 'F': []}
def ucs(start,goal):
    q=[(0,start)]
    visited=set()
    while q:
        cost,node=heapq.heappop(q)
        if node==goal:
            print(cost);return
        if node not in visited:
            visited.add(node)
            for n,w in graph[node]:
                heapq.heappush(q,(cost+w,n))
ucs('A','D')
```

Question: 4. Basic Medical Diagnosis Rule-Based Expert System.

```
fever(john).
cough(john).
headache(john).
disease(X,flu):-fever(X),cough(X),headache(X).

Query:
?- disease(john,D).
```

Question: 5. Simple Path-Finding Logic.

```
path(a,b).
path(b,c).
path(c,d).
connected(X,Y):-path(X,Y).
```

```
Query:
?- connected(a,b).
```

Question: 6. Animal Identification.

```
animal(cat).
animal(dog).
sound(cat,meow).
sound(dog,bark).
```

```
Query:
?- sound(cat,X).
```

Question: 7. Family Relationships.

```
parent(ali,ahmed).
parent(ahmed,sara).
grandparent(X,Y):-parent(X,Z),parent(Z,Y).
```

```
Query:
?- grandparent(ali,sara).
```

Question: 8. Factorial.

```
factorial(0,1).
factorial(N,F):-N>0,N1 is N-1,factorial(N1,F1),F is N*F1.
```

```
Query:
?- factorial(5,X).
```

Question: 9. Student Pass or Fail.

```
result(M,pass):-M>=35.
result(M,fail):-M<35.
```

```
Query:
?- result(60,X).
```

Question: 10. Vote Eligibility.

```
eligible(A):-A>=18.
not_eligible(A):-A<18.
```

```
Query:
?- eligible(20).
```

Question: 11. Even or Odd.

```
even(X):-0 is X mod 2.
odd(X):-1 is X mod 2.
```

```
Query:
?- even(8).
```

Question: 12. Adult or Minor.

```
adult(A):-A>=18.
minor(A):-A<18.
```

```
Query:
?- adult(22).
```

Question: 13. Distinction, Pass or Fail.

```
result(M,distinction):-M>=75.  
result(M,pass):-M>=35,M<75.  
result(M,fail):-M<35.
```

```
Query:  
?- result(80,X).
```

Question: 14. Positive, Negative or Zero.

```
positive(X):-X>0.  
negative(X):-X<0.  
zero(0).
```

```
Query:  
?- positive(5).
```

Question: 15. Driving License Eligibility.

```
license(A):-A>=18.  
not_license(A):-A<18.
```

```
Query:  
?- license(21).
```

Question: 16. Grade Based on Marks.

```
grade(M,'A'):-M>=75.  
grade(M,'B'):-M>=60,M<75.  
grade(M,'C'):-M>=35,M<60.  
grade(M,'F'):-M<35.
```

```
Query:  
?- grade(65,X).
```

Question: 17. Divisible by 5.

```
divisible_by_5(X):-0 is X mod 5.
```

```
Query:  
?- divisible_by_5(25).
```